Institut Teknologi Bandung

Directorate of Sustainable Professional Education

CHAPTER 2

The Mathematical Building Blocks of Neural Networks

Tensors, tensor operations, and gradient-based optimisation — explained through code you can run rather than notation you have to decode.

Duration 3 hours (2 sessions)

Instructor Rahman Indra Kesuma, S.Kom., M.Cs. · Prof. Bambang Riyanto Trilaksono

Source Chollet & Watson, Deep Learning with Python 3e — chapter 2

LEARNING OUTCOMES

Session Objectives

By the end of this session, participants can:

1. Run the first MNIST example end to end and name the role of **layers, loss, optimizer, and metrics** in each line of it.
2. Read a tensor's **rank, shape, and dtype**, and map real data — vectors, timeseries, images, video — onto its tensor shape.
3. Explain **element-wise operations, broadcasting, the tensor product, and reshaping**, together with what each one means geometrically.
4. Describe **derivatives, gradients, mini-batch SGD, and the chain rule**, and point to where each lives in a computation graph.
5. Rewrite MNIST **from scratch** — layer, model, batch generator, and training loop — without calling `fit()`.

BOOK
Chapter 2 — full text

NOTEBOOK
01_first_mnist.ipynb

NOTEBOOK
02_tensors_and_operations.ipynb

NOTEBOOK
03_gradients_and_sgd.ipynb

NOTEBOOK
04_mnist_from_scratch.ipynb

NOTEBOOK
All chapter 2 notebooks

REPO
Author's official notebook

ROADMAP

One example, taken apart all the way down

The chapter travels a full circle: run MNIST with `fit()`, dismantle every piece of it, then **rewrite the whole thing from scratch** and show the result holds up.

1 · First example
MNIST in ten lines. Working first, understood second.
2.1

2 · Tensors
Rank, shape, dtype, slicing, the batch axis, real-world data.
2.2

3 · Tensor operations
Element-wise, broadcasting, matmul, reshape — and their geometry.
2.3

4 · The engine
Derivatives, gradients, SGD, the chain rule, autodiff.
2.4 – 2.6

ROADMAP

Why this chapter shows code instead of notation

> *Runnable code is the most precise, unambiguous description of a mathematical operation.*
>
> The working principle of chapter 2

Every concept here is introduced twice: once as an idea, and once as a few lines you can execute and inspect. If the two ever disagree, **the code is right**

01

A first look at a neural network

MNIST — the "hello world" of deep learning.

SECTION 2.1

The data: 60,000 to train on, 10,000 to be judged on

listing 2.1 — loading MNIST PYTHON

```python
from keras.datasets import mnist

(train_images, train_labels), (test_images, test_labels) = mnist.load_data()

print(train_images.shape, train_images.dtype)
print(len(train_labels), train_labels[:10])
print(test_images.shape)
```

OUTPUT

```
(60000, 28, 28) uint8
60000 [5 0 4 1 9 2 1 3 1 4]
(10000, 28, 28)
```

SECTION 2.1

Three words used precisely from here on

TERM	WHAT IT MEANS HERE
Sample	One data point — a single 28×28 image.
Class	One category — a digit from 0 to 9.
Label	The class attached to one particular sample.

These are used consistently for the rest of the book, and mixing them up is the fastest way to misread an error message later.

SECTION 2.1

The model: two layers, and what a layer is for

listing 2.2 — the model PYTHON

```python
import keras
from keras import layers

model = keras.Sequential([
    layers.Dense(512, activation="relu"),
    layers.Dense(10, activation="softmax"),
])
```

The final `softmax` layer emits **10 probability scores that sum to 1** — one per digit class. Note there is no `input_shape` anywhere; Keras infers it on the first call, which chapter 3 explains.

SECTION 2.1

Compilation: three choices, and only three

listing 2.3 — compile PYTHON

```python
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)
```

Loss

The feedback signal the network steers by. **This is what gets minimised.**

Optimizer

The mechanism by which the network updates itself from that signal.

Metrics

Watched during training, but **never optimised for**.

SECTION 2.1

Preprocessing: flatten, cast, and scale

listing 2.4 — preparing the data PYTHON

```python
train_images = train_images.reshape((60000, 28 * 28))
train_images = train_images.astype("float32") / 255

test_images = test_images.reshape((10000, 28 * 28))
test_images = test_images.astype("float32") / 255

print(train_images.shape, train_images.dtype, train_images.min(), train_images.max())
```

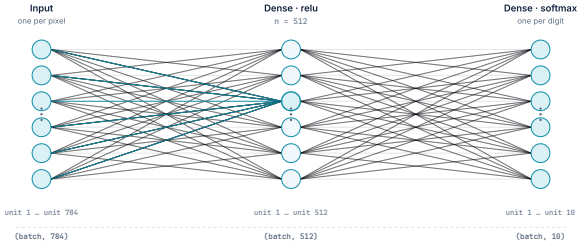
OUTPUT

```
(60000, 784) float32 0.0 1.0
```

Two things changed: the shape went from `(60000, 28, 28)` to `(60000, 784)`, and the values went from `0..255` integers to `0..1` **floats**. Chapter 6 explains why the second matters so much.

SECTION 2.1

The same network, drawn as what it is

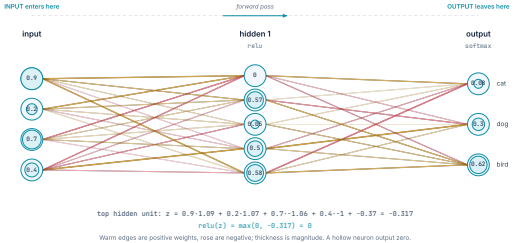


784 units, then 512, then 10. Six are drawn per layer and the rest are the ellipsis — the counts underneath are the real ones.

This is the model from listing 2.2, drawn. Every unit in a layer is connected to every unit in the next, which is what `Dense` means — and why the first layer alone holds **401,920 weights**. The next slide shrinks it to four inputs so the arithmetic fits on a slide.

SECTION 2.1

One forward pass, with the arithmetic



A four-input toy of the same shape, small enough that every number fits on the slide. Press play.

SECTION 2.1

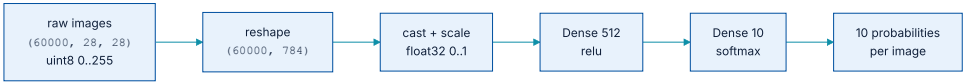
Reading the forward pass

1. **Every edge is a weight, and the drawing shows it.** Warm means positive, rose means negative, and thickness is magnitude. A trained network is exactly this picture with different numbers on the edges — nothing else changes.
2. **Each unit sums what reaches it, adds a bias, then applies relu.** The sum for the top hidden unit is printed under the figure so you can check it against the inputs and weights shown.
3. **A unit reading `0` is not broken — it is `relu` doing its job.** Its sum came out negative and `max(0, z)` clipped it. That unit contributes nothing to this input, and **a different input would wake it up**.
4. **The output layer uses softmax**, so its three numbers are positive and sum to 1 — which is what makes them readable as probabilities.

> Scale this to 784 inputs, 512 hidden units and 10 outputs and you have the MNIST model from listing 2.2 — **401,920 weights on the first layer alone**, every one doing what the four here are doing.

SECTION 2.1

The whole first example, as a shape pipeline



Every arrow changes either the shape or the dtype. Nothing else happens in the first example.

SECTION 2.1

Training, and the first crack to notice

listing 2.5 — fit PYTHON

```python
model.fit(train_images, train_labels, epochs=5, batch_size=128)

test_loss, test_acc = model.evaluate(test_images, test_labels)
print(f"test accuracy: {test_acc:.3f}")
```

OUTPUT

```
Epoch 1/5
469/469 ---- 3s 5ms/step - accuracy: 0.8747 - loss: 0.4358
Epoch 5/5
469/469 ---- 2s 5ms/step - accuracy: 0.9890 - loss: 0.0361

313/313 ---- 1s 2ms/step - accuracy: 0.9780 - loss: 0.0745
test accuracy: 0.978
```

98.9% on data it trained on, **97.8%** on data it had never seen. That gap is not noise — it is **overfitting**, and chapter 5 treats it as the central problem of the field.

SECTION 2.1

Making a prediction, and reading it

listing 2.6 — predicting PYTHON

```python
test_digits = test_images[0:10]
predictions = model.predict(test_digits)

print(predictions[0].argmax())      # which class?
print(predictions[0].max())         # how confident?
print(test_labels[0])               # what was the truth?
```

OUTPUT

```
7
0.99993
7
```

Right answer, and near-total confidence. Chapter 5 will show why **confidence and correctness are not the same thing**

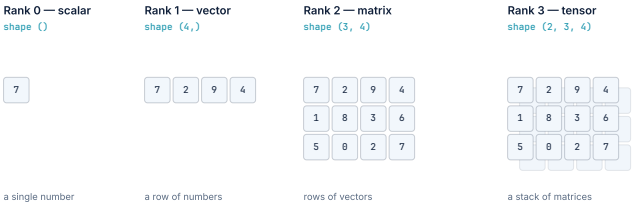
02

Data representations: tensors

A container for data. Three attributes, and one special axis.

SECTION 2.2

Rank, shape, dtype



Rank is how many indices it takes to reach one number: t, t[i], t[i][j], t[i][j][k].

The same four numbers 7, 2, 9, 4 appear at every rank. What changes is how many indices it takes to reach one of them.

SECTION 2.2

Reading the drawing: what each rank actually is

RANK	NAME	SHAPE	TO REACH ONE NUMBER	EXAMPLE
0	scalar	`()`	`t`	a loss value — one number per batch
1	vector	`(4,)`	`t[i]`	one MNIST image after flattening is `(784,)`
2	matrix	`(3, 4)`	`t[i][j]`	a batch of flattened images, `(128, 784)`
3	tensor	`(2, 3, 4)`	`t[i][j][k]`	a batch of images before flattening, `(128, 28, 28)`

Rank is a count of axes, not a size. A `(60000, 28, 28)` tensor and a `(2, 3, 4)` tensor are both rank 3, and every rule you learn about one applies to the other. **That is the whole reason the abstraction is worth having.**

> The word *dimension* is used for two different things and causes real confusion: a rank-3 tensor has **3 axes**, while a `(784,)` vector is often called *784-dimensional*. Say **rank** for axes and **shape** for sizes, and the ambiguity disappears.

SECTION 2.2

Reading them off a real tensor

the three attributes

PYTHON

```python
(train_images, train_labels), _ = mnist.load_data()

print(train_images.ndim)     # rank: how many axes
print(train_images.shape)    # shape: how long each axis is
print(train_images.dtype)    # dtype: what the entries are
```

OUTPUT

```
3
(60000, 28, 28)
uint8
```

SECTION 2.2

The terminology trap that costs people an afternoon

> A **5-dimensional vector** is not a **5-dimensional tensor**. The first has **one axis holding five numbers**; the second has **five axes**

Misreading this makes shape error messages unintelligible, and shape errors are the most common failure you will hit in the exercises.

SECTION 2.2

Slicing tensors

listing 2.7–2.8 — slices

PYTHON

```python
print(train_images[10:100].shape)          # 90 images
print(train_images[:, 14:, 14:].shape)     # bottom-right 14x14 corner
print(train_images[:, 7:-7, 7:-7].shape)   # centre 14x14, via negative indices
```

OUTPUT

```
(90, 28, 28)
(60000, 14, 14)
(60000, 14, 14)
```

SECTION 2.2

The batch axis is always axis 0

listing 2.9 — batches PYTHON

```python
batch = train_images[:128]        # batch 0
batch = train_images[128:256]     # batch 1

n = 3
batch = train_images[128 * n : 128 * (n + 1)]     # batch n
print(batch.shape)
```

OUTPUT

```
(128, 28, 28)
```

Axis 0 is the **samples axis** — and because of batching it is also called the **batch axis**. **Every shape error you will read starts with identifying this axis.**

Real-world data, and the shape it takes

KIND OF DATA	RANK	SHAPE	EXAMPLE FROM THE BOOK
Vector data	2	`(samples, features)`	100,000 people × (age, gender, income) → (100000, 3)
Timeseries	3	`(samples, timesteps, features)`	250 days × 390 minutes × 3 values → (250, 390, 3)
Images	4	`(samples, h, w, channels)`	128 RGB images at 256×256 → (128, 256, 256, 3)
Video	5	`(samples, frames, h, w, channels)`	4 clips × 240 frames × 144×256 × RGB → (4, 240, 144, 256, 3)

SECTION 2.2.9

Channels-last or channels-first — and why it bites

- **Channels-last** `(..., h, w, c)` — the TensorFlow and JAX convention.
- **Channels-first** `(..., c, h, w)` — the PyTorch convention.

Keras 3 picks between them with `image_data_format`.

> Getting this wrong produces shape errors that look nonsensical, because both orderings are *valid* — just not for the same layer.

106,168,320
values in that 60-second video example

425 MB
its size at float32

03

The gears: tensor operations

It all reduces to a handful of operations — each with a geometric meaning.

SECTION 2.3

A Dense layer is three operations, and nothing else

the whole of a Dense layer

PYTHON

```
output = relu(matmul(input, W) + b)
#        ^           ^             ^
#        |           |             +-- addition (with broadcasting)
#        |           +---------------- tensor product
#        +---------------------------- element-wise operation
```

`relu(x)` is simply `max(x, 0)`: it zeroes out anything negative and leaves the rest alone.

SECTION 2.3.1

Element-wise operations, written the slow way

listing 2.10 — naive relu

PYTHON

```python
def naive_relu(x):
    assert len(x.shape) == 2
    x = x.copy()
    for i in range(x.shape[0]):
        for j in range(x.shape[1]):
            x[i, j] = max(x[i, j], 0)
    return x
```

Correct, readable — and unusably slow, as the next slide shows.

. . . and the fast way, which is the same operation

the vectorised version

PYTHON

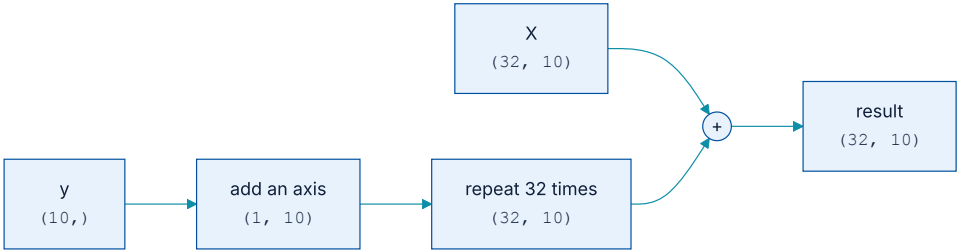
```
z = x + y
z = np.maximum(z, 0.0)
```

0.02 s
vectorised NumPy, 1,000 iterations

2.45 s
naive Python loops, 1,000 iterations

Roughly **100**×, and it is not a language effect. NumPy hands the work to BLAS routines written in C and Fortran; on a GPU the same operation runs as **fully vectorised CUDA**

Broadcasting: fitting a small shape to a big one



Two steps: add axes until the ranks match, then repeat along the new axes.

No memory is actually duplicated — the repetition is **algorithmic, not physical**

SECTION 2.3.2

The rule, and where it quietly goes wrong

listing 2.11 — broadcasting in anger PYTHON

```python
X = np.random.random((64, 3, 32, 10))
y = np.random.random((32, 10))

z = np.maximum(X, y)      # y is broadcast across the first two axes
print(z.shape)
```

OUTPUT

```
(64, 3, 32, 10)
```

> This is the most common source of **silent** bugs: the shapes line up, the code runs, and the **meaning is wrong**. When a result looks strange, print the shapes first.

SECTION 2.3.3

The tensor product, and its one compatibility rule

matmul PYTHON

```python
z = np.matmul(x, y)
z = x @ y                       # the same thing, shorter

# compatibility:  x.shape[1] == y.shape[0]
# result shape:  (x.shape[0], y.shape[1])

# (a, b, c, d) @ (d,)     -> (a, b, c)
# (a, b, c, d) @ (d, e)  -> (a, b, c, e)
```

Visually: line the two up as rectangles — **the width of the first must match the height of the second**

SECTION 2.3.4

Reshaping rearranges; it never invents or destroys

listing 2.12 — reshape and transpose PYTHON

```python
x = np.array([[0., 1.],
              [2., 3.],
              [4., 5.]])          # shape (3, 2)

print(np.reshape(x, (6,)).shape)     # flattened
print(np.reshape(x, (2, 3)).shape)   # regrouped

print(np.transpose(np.zeros((300, 20))).shape)   # rows and columns exchanged
```

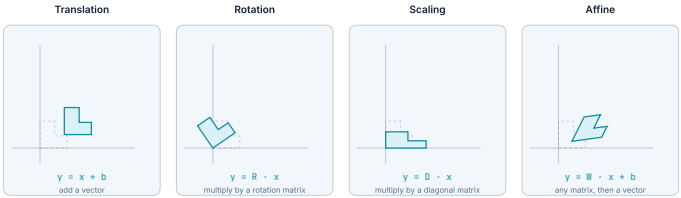
OUTPUT

```
(6,)
(2, 3)
(20, 300)
```

This is exactly what `train_images.reshape((60000, 784))` did in the first example: **the pixels never changed, only their arrangement**

SECTION 2.3.5

Every tensor operation is a geometric one



Dashed: before. Solid: after. Every layer in a network does the last one — a matrix, then a vector.

One shape, four operations. The dashed outline is where it started.

SECTION 2.3.5

Reading the drawing: what moved, and what did it

OPERATION	WHAT IT DOES TO THE SHAPE	WRITTEN AS
Translation	Slides it. Every point moves by the same vector; nothing turns or stretches.	`y = x + b`
Rotation	Turns it about the origin. Lengths and angles survive — the shape is the same shape, pointing elsewhere.	`y = R @ x`
Scaling	Stretches each axis by its own factor. Here x by 1.5 and y by 0.6, which is why it comes out squat.	`y = D @ x`
Affine	Any matrix, then a vector. Straight lines stay straight and parallel lines stay parallel — but angles do not survive.	`y = W @ x + b`

A `Dense` layer is the fourth row. `W @ x + b` is an affine transform, and nothing else. A stack of them with no activation between is still one affine transform — which is the next slide, and the reason activations exist.

SECTION 2.3.5

Why an activation function is not optional

two affine layers collapse into one — PYTHON

```
affine2(affine1(x)) == (W2 @ W1) @ x + (W2 @ b1 + b2)
#                       \________/         \_____________/
#                        one matrix           one vector
```

So a deep stack of `Dense` layers with no activation is **secretly a single linear model**, however many layers you give it. Nonlinearities such as `relu` are what make the hypothesis space rich.

SECTION 2.3.5

Deep learning as uncrumpling a paper ball

> It happens through a long series of **small, elementary geometric moves** — like uncrumpling a paper ball with successive finger movements. **Each layer disentangles the data a little.**

Chapter 5 makes this precise, under the name of the *manifold hypothesis*.

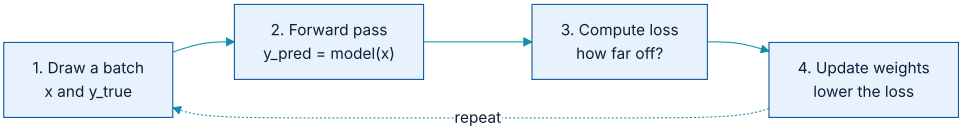
04

The engine: gradient-based optimisation

Derivatives, gradients, SGD, and the chain rule.

SECTION 2.4

The training loop, and which step is hard



Four steps, repeated over batches. Only the fourth is difficult.

> Tuning each coefficient by hand is impossible — modern networks have **millions to billions of parameters**. Gradient descent solves for all of them at once.

SECTION 2.4.1

A derivative is the slope of a local approximation

- `a` negative → increasing `x` **decreases** `f(x)`.
- `a` positive → increasing `x` **increases** `f(x)`.
- Its magnitude says how fast the change happens.

> So to make `f` smaller, move `x` **in the direction opposite its derivative**. That single sentence is the whole of optimisation.

SECTION 2.4.2

A gradient is a derivative for tensors

what the gradient is a gradient of

PYTHON

```
y_pred = matmul(x, W)
loss_value = loss(y_pred, y_true)

# For fixed x and y_true this is just  loss_value = f(W)
# grad(loss_value, W0) is a tensor SHAPED LIKE W, whose every coefficient says
# in which direction, and how strongly, the loss moves if you nudge that weight.
```

To lower the loss, step against the gradient: `W1 = W0 - step * grad(f(W0), W0)`

SECTION 2.4.3

Why this is done by iteration and not by solving

Because for a network with millions of parameters that is **analytically intractable**. Iteration is not a shortcut; it is the only available route.

SECTION 2.4.3

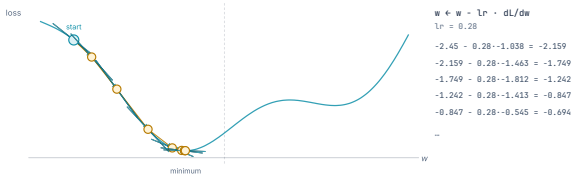
Mini-batch stochastic gradient descent

1. Draw a **random** batch of samples and targets. (The word *stochastic* comes from this randomness.)
2. Forward pass → predictions.
3. Compute the loss.
4. **Backward pass** → gradient of the loss with respect to the parameters.
5. `W -= learning_rate * gradient`.

Repeat. Each step lowers the loss a little; enough steps and the model fits.

SECTION 2.4.3

Seven steps of it, on a real curve



The step shrinks on its own as the slope flattens — nothing in the rule schedules that.

Cyan: the slope measured at that point. Amber: the step taken because of it.

SECTION 2.4.3

Reading the walk: three things to notice

1. **The step is the slope, scaled.** Nothing decides how far to move except `learning_rate × gradient`. Where the curve is steep the step is long; where it flattens the step shortens **on its own**. No schedule does that — it falls out of the rule.
2. **It never lands exactly on the minimum,** and it does not need to. Training stops when the loss stops improving, not when the gradient reaches zero.
3. **Only the local slope is known.** At every point the walk sees the tangent and nothing else — not the shape of the curve, not where the minimum is. That is why a bad learning rate is fatal: the step is taken blind.

The arithmetic beside the curve is the whole algorithm. `w ← w − lr · dL/dw`, five times, with real numbers you can check on paper. **Everything else in this chapter is about computing that one derivative efficiently** for millions of parameters at once.

SECTION 2.4.3

Learning rate: the one number that decides whether it works

Too small

Convergence crawls — many steps for very little progress. It can look as though training has stalled.

Momentum

Update using the current gradient **and** previous updates — like a ball rolling downhill with enough speed to cross a shallow dip.

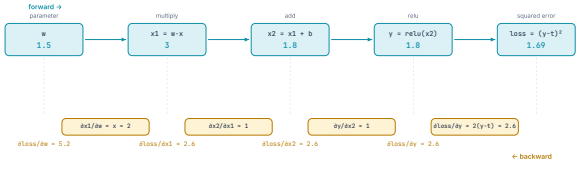
Too large

Updates overshoot wildly; the loss jumps around and **never comes down**.

The variants that improve on plain SGD are called **optimizers**: SGD with momentum, Adagrad, RMSprop, Adam.

SECTION 2.4.4

The chain rule, applied backwards over a graph



Top row forward, bottom row backward. Every number computed from the one above it.

SECTION 2.4.4

Reading it backwards: three things that make it work

1. **Each node only needs its own derivative.** Multiply by 1 for the addition, by `x` for the multiplication, by 1 or 0 for the relu. Nothing on the slide required knowing the whole graph — which is exactly why this scales to a network with millions of nodes.
2. **The gradient is a product along the path.** `2.6 × 1 × 1 × 2 = 5.2`. Applying the chain rule this way, node by node from the loss backwards, **is** backpropagation. There is no further mechanism.
3. **The forward values are needed by the backward pass.** `∂x1/∂w` is `x`, so the input has to still be there when the gradient comes back. That is why training uses so much more memory than inference — the forward pass is kept.

Now make the relu die. Set `b = -1.2` so `x2` comes out negative. Then `∂y/∂x2 = 0`, the product collapses, and `∂loss/∂w = 0` — the weight receives **no** gradient and cannot learn from this example. A dead unit is not a bug in the code; it is this multiplication reaching a zero.

SECTION 2.4.5

Two details that make the graph work

- When there are **several paths** from node `a` to node `b`, the contributions of the paths are **summed**.
- A **computation graph** is a directed acyclic graph of operations. It makes computation itself into data — a structure a machine can read and manipulate.

> Modern frameworks walk that graph for you. That is **automatic differentiation**, and it is why **you will never write backpropagation by hand**

SECTION 2.4.5

Autodiff in three lines

listing 2.19 — a first derivative PYTHON

```python
import tensorflow as tf

x = tf.Variable(3.0)
with tf.GradientTape() as tape:
    y = 2 * x + 3

print(tape.gradient(y, x))      # dy/dx
```

OUTPUT

```
tf.Tensor(2.0, shape=(), dtype=float32)
```

The answer is 2, which is what you would get by hand — except nothing here was told the rule for differentiating a line.

SECTION 2.4.5

. . . and a second derivative, which is a physics answer

listing 2.20 — nested tapes PYTHON

```
time = tf.Variable(0.0)

with tf.GradientTape() as outer_tape:
    with tf.GradientTape() as inner_tape:
        position = 4.9 * time ** 2
    speed = inner_tape.gradient(position, time)

acceleration = outer_tape.gradient(speed, time)
print(acceleration)
```

OUTPUT

```
tf.Tensor(9.8, shape=(), dtype=float32)
```

`position = 4.9·t²` is free fall. Its second derivative is the **acceleration due to gravity**, and autodiff returned 9.8 without ever being given the formula.

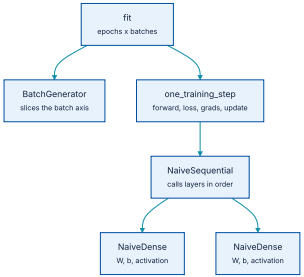
05

Rewriting the first example from scratch

No fit(), no built-in Dense. The result has to hold up.

SECTION 2.6

What we are about to rebuild



Four small classes and one function replace Sequential, Dense, and fit() between them.

SECTION 2.6.1

A layer, written out

listing 2.21 — NaiveDense PYTHON

```python
import keras
from keras import ops

class NaiveDense:
    def __init__(self, input_size, output_size, activation=None):
        self.activation = activation
        self.W = keras.Variable(shape=(input_size, output_size), initializer="uniform")
        self.b = keras.Variable(shape=(output_size,), initializer="zeros")

    def __call__(self, inputs):
        x = ops.matmul(inputs, self.W) + self.b
        return self.activation(x) if self.activation is not None else x

    @property
    def weights(self):
        return [self.W, self.b]
```

`W` starts random and `b` starts at zero — exactly the starting condition chapter 1 described.

SECTION 2.6.1

A model is just layers, applied in order

listing 2.22 — NaiveSequential PYTHON

```python
class NaiveSequential:
    def __init__(self, layers):
        self.layers = layers

    def __call__(self, inputs):
        x = inputs
        for layer in self.layers:
            x = layer(x)
        return x

    @property
    def weights(self):
        return [w for layer in self.layers for w in layer.weights]
```

Note how little there is to it: a model is **a list of layers and a for loop**. Everything else in Keras is convenience on top.

One training step is the four figures, in code

listing 2.24 — one_training_step PYTHON

```python
import tensorflow as tf
from keras import optimizers

optimizer = optimizers.SGD(learning_rate=1e-3)

def one_training_step(model, images_batch, labels_batch):
    with tf.GradientTape() as tape:
        predictions = model(images_batch)                       # forward
        loss = ops.sparse_categorical_crossentropy(labels_batch, predictions)
        average_loss = ops.mean(loss)                           # loss
    gradients = tape.gradient(average_loss, model.weights)      # gradients
    optimizer.apply_gradients(zip(gradients, model.weights))    # update
    return average_loss
```

Four lines, four ideas: **forward, loss, gradients, update**. Every training loop in the rest of the book is a variation on this shape.

SECTION 2.6.4

And the loop around it

listing 2.26 — fit, by hand

PYTHON

```python
def fit(model, images, labels, epochs, batch_size=128):
    for epoch in range(epochs):
        print(f"Epoch {epoch}")
        gen = BatchGenerator(images, labels, batch_size)
        for i in range(gen.num_batches):
            images_batch, labels_batch = gen.next()
            loss = one_training_step(model, images_batch, labels_batch)
            if i % 100 == 0:
                print(f"  loss at batch {i}: {loss:.2f}")
```

`BatchGenerator` is a dozen lines that hand out consecutive slices — the batch axis from section 2.2, put to work.

SECTION 2.6.5

Does it work? Run it and see

listing 2.27 — evaluating it PYTHON

```python
model = NaiveSequential([
    NaiveDense(input_size=28 * 28, output_size=512, activation=ops.relu),
    NaiveDense(input_size=512, output_size=10, activation=ops.softmax),
])
fit(model, train_images, train_labels, epochs=10, batch_size=128)

predictions = model(test_images)
predicted_labels = ops.argmax(predictions, axis=1)
print(f"accuracy: {ops.mean(predicted_labels == test_labels):.2f}")
```

OUTPUT

```
Epoch 0
  loss at batch 0: 6.19
  loss at batch 400: 2.21
Epoch 9
  loss at batch 0: 0.36
  loss at batch 400: 0.34
accuracy: 0.90
```

SECTION 2.6.5

90% versus 97.8% — and why that is the lesson

> The hand-written version scores **lower**, and that is not a bug. It uses plain SGD at `learning_rate=1e-3`; the first example used **Adam**

So the gap measures **the choice of optimizer**, not the quality of the implementation. It is one of the clearest demonstrations in the book of how much that choice is worth.

SUMMARY

What has to survive this chapter

1. **Tensors** are containers with **rank, shape, dtype**. Axis 0 is the samples axis, and therefore the batch axis.
2. **Tensor operations are geometric.** A Dense layer without an activation is an affine transform, and stacking those still gives an affine transform.
3. **Learning** means finding weight values that minimise the loss on the training data.
4. **Mini-batch SGD** updates weights from gradients computed on random batches.
5. **Chain rule + autodiff = backpropagation**, carried out over a computation graph.
6. **Loss** measures success at the task; **optimizer** decides how the descent is performed — and that choice is worth several points of accuracy.

NOTEBOOK
01_first_mnist.ipynb

NOTEBOOK
04_mnist_from_scratch.ipynb

NEXT
Chapter 3 — The frameworks